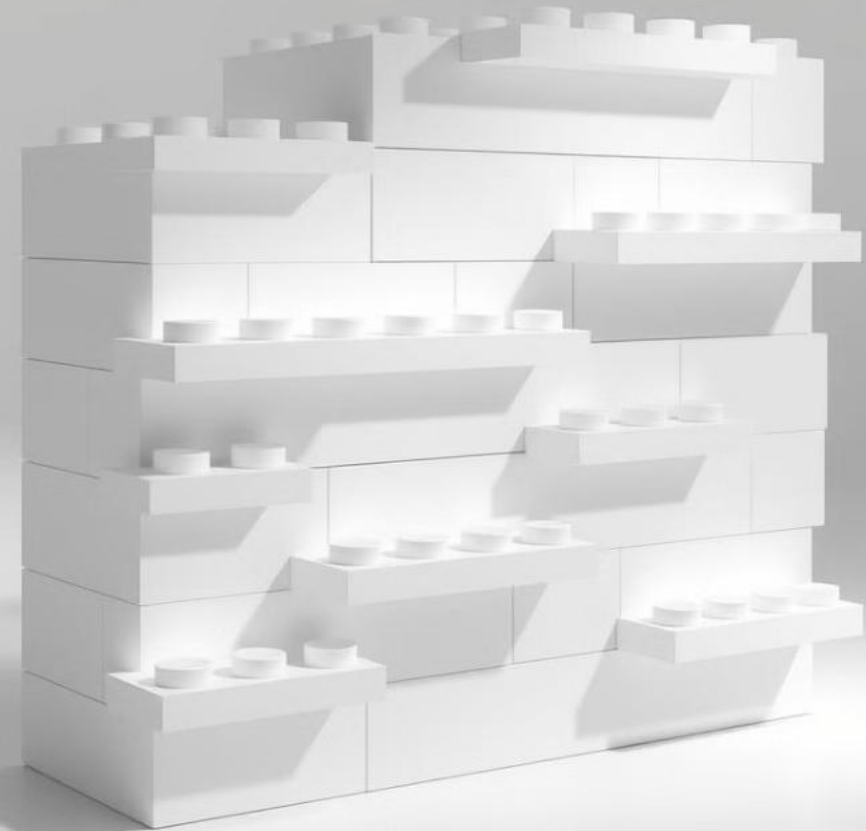


INTRODUCTION TO COMPONENT BASED SOFTWARE DEVELOPMENT

CHAPTER 5 – COMPONENT MODELS WITH ARCHITECTURAL UNITS AS COMPONENTS



Contents

- 1. Acme**
- 2. ArchJava**
- 3. UML**
- 4. Domain Specific Models**
- 5. Others**

CMs with Architectural Units as Components

- here components are *architectural units*, and
- their models are described by *Architecture Description Languages (ADLs)*
- The archetypal example is *Acme*

Architectural unit

- ADLs define software architecture (visible parts / sub-systems / *components* / Architectural Units *and their inter-relationship*)
 - each part (*component*) - is called an *Architectural Unit*
 - this is a Unit of behavior/data
- each Architectural Unit has *input* and *output ports*
 - and their ports are linked together by *connectors* which can convey data/control
 - this defines their inter-relationship

System starts from a *main method* either in

- one of the Architectural units or
- from a client application that invokes one of them
- architectural units connect and collaborate via these connectors through ports (*indirect method invocation*) to provide services

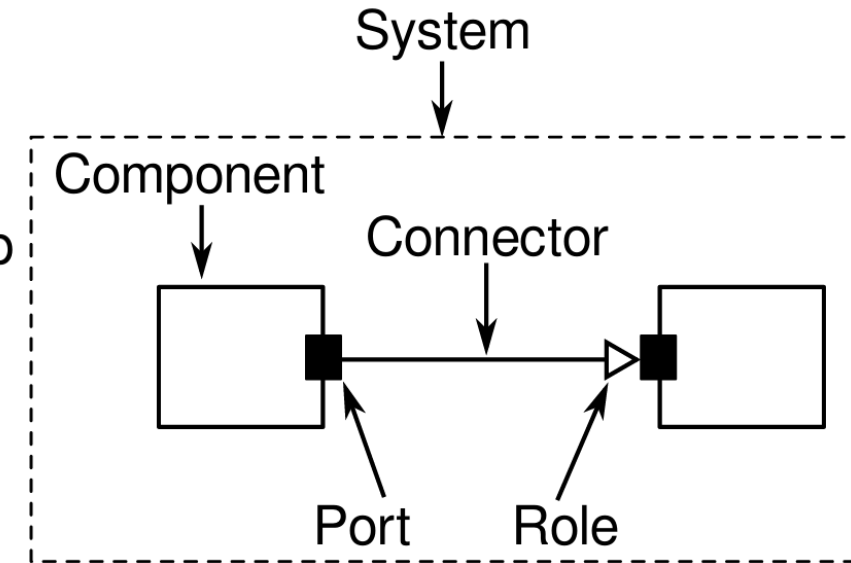
5.1 Acme

- in its entirety Acme is more than an ADL, its a meta model for ADL, so its foundations and underlying concepts underpin all ADLs. But here we discuss the core elements of Acme for modeling



Component ATM = { Port receiveReq, Port sendReq }

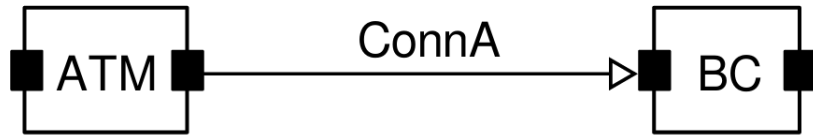
Component BC = { Port receiveOp, Port sendOp }



Components- primary units of computation and data

- a components has multiple interfaces (**ports**)
- interface could be a simple procedure signature or a collection of procedures to invoke in a certain sequence
- components are composed by **connectors** between them

5.1 Acme



Component ATM = { Port receiveReq, Port sendReq }

Component BC = { Port receiveOp, Port sendOp }

Connector ConnA = { **Roles** {request, produce} }

Attachments : {

ATM.sendReq to ConnA.request;

ConnA.produce to BC.receiveOp;

}

- **connectors** define a set of **roles** based on the type of interaction (RPC, event broadcast, pipe,...)
 - in event broadcast we have one source role and many event-receiver roles
 - in RPC there is a caller and a callee
- connections are specified in **attachments**, i.e. ports are attached to their roles in interaction

5.1 Acme

System BankSys = {

Component ATM = { Port receiveReq, Port sendReq }

Component BC= { Port receiveOp, Port sendOp}

Component Bank1 { Port receiveOp, Port sendRes, Property bankId: String}

Component Bank2 { Port receiveOp, Port sendRes, Property bankId: String}

Connector ConnA = {Roles {request, produce}}

Connector ConnB = {Roles {request, produce}}

Connector ConnC = {Roles {request, produce}}

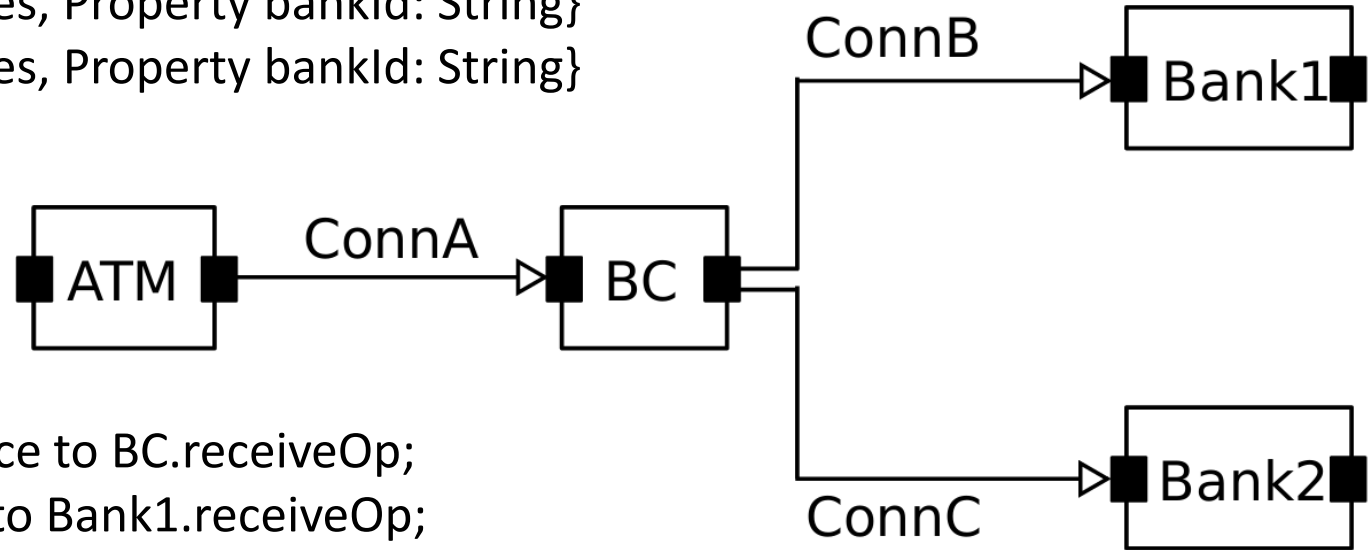
Attachments: {

ATM.sendReq to ConnA.request; ConnA.produce to BC.receiveOp;

BC.sendOp to ConnB.request; ConnB.produce to Bank1.receiveOp;

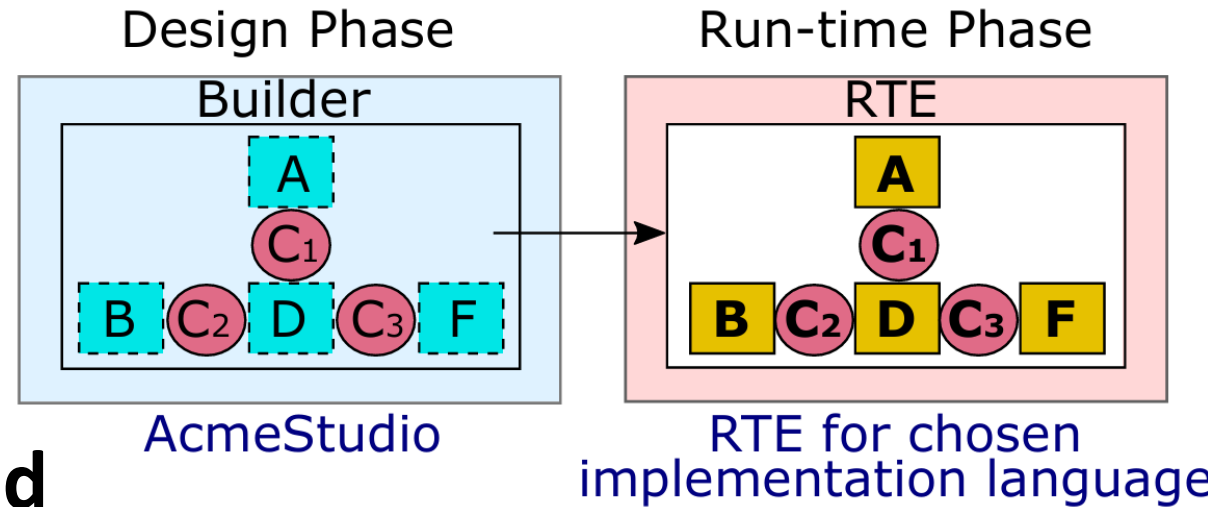
BC.sendOp to ConnC.request; ConnC.produce to Bank2.receiveOp;

}}



Systems- application architecture represented as graph of **components** and lines denoting **connectors** linking ports – expressed in **attachments**

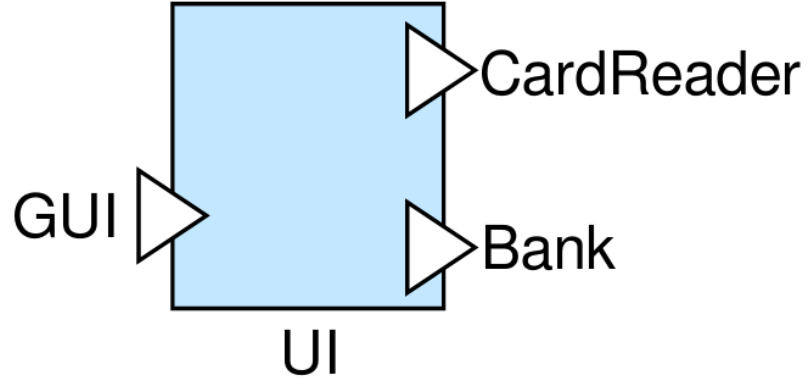
5.1 Acme



Relation to Idealized

- Design Phase:
 - **system** modeled in terms of **components** and **connectors** (**AcmeStudio**)
 - since it's only a model, there is no code, and thus **no repository**
 - programmer implements these in the language of their choice (like Java)
- Deployment Phase: **no deployment phase**
- Run-time Phase: run-time environment of implementing language

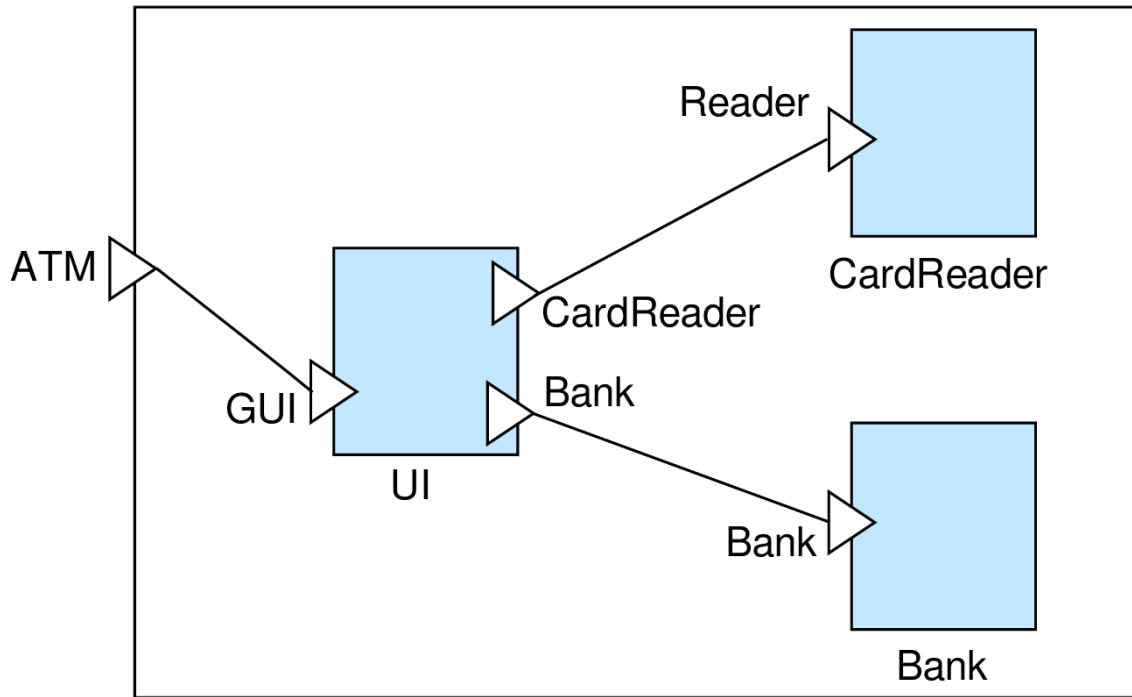
5.2 ArchJava



```
public component class UI{  
    public port GUI{  
        provides void display(); }  
    public port Bank{  
        requires void deposit(String accno, Double amount);  
        requires void withdraw(String accno, Double amount);  
        requires Double balance(String accno); }  
    public port CardReader{  
        requires String readAccNo(); }  
}
```

- an ADL based on acme focused on only defining models
- it's a programming language for defining **components** and **architectures**
 - extends Java with the special **component** class for defining architectural units
 - each component has **ports**, specifying its **required** and **provided** services
 - composition happens by linking these ports forming the **architecture**
 - components have multiple subsystems at layers - enabling **multi-level composition**

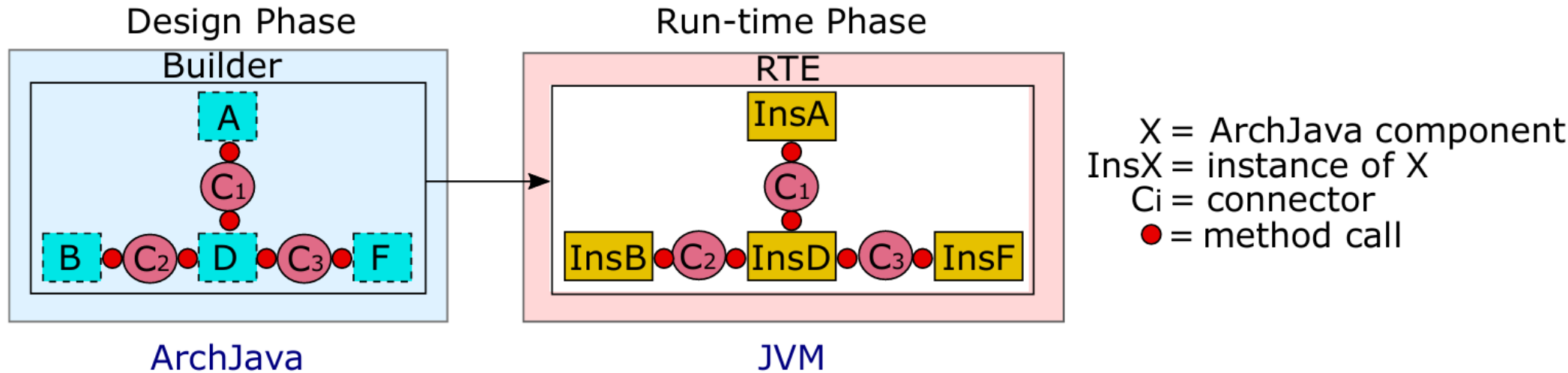
5.2 ArchJava > Composition Example



```
public component class ATMSystem{  
  public port ATM{  
    provides void start(); }  
  // Define sub-components  
  private UI gui = new UI();  
  private Bank bank = new Bank();  
  private CardReader cardReader = new CarReader();  
  // Compose sub-components  
  connect gui.CardReader, cardReader.Reader;  
  connect gui.Bank, bank.Bank;  
  glue this.ATM, gui.GUI;  
}
```

- **Composite:** ATM providing GUI and requiring CardReader, Bank - services
- ATM composite containing the three components and its code for the entire ATM (parts are **connected**, and **glued** to parent)
- **main method of system** should be in the top-level component class

5.2 ArchJava

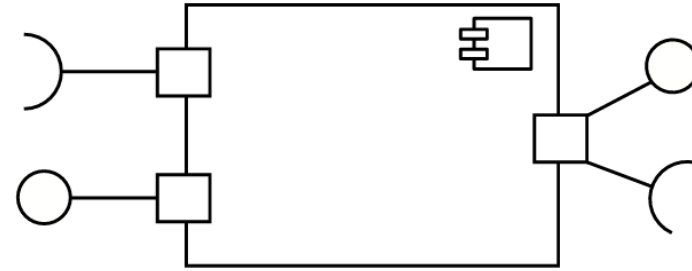


Relation to Idealized

- Design Phase:
 - system modeling and coding for all components and connections in ArchJava
 - all connections are through **connectors** - *indirect method call*
- Deployment Phase: ***no deployment phase*** and **no repository**
- Run-time Phase: - code is compiled and executed in JVM at run-time, where components are instances of ArchJava component class defined at design phase

5.3 UML 2.x

- starting from UML 2.0, UML 2.x provides an ADL for defining components and architecture (Just like Acme), it was called ***structured classifier***

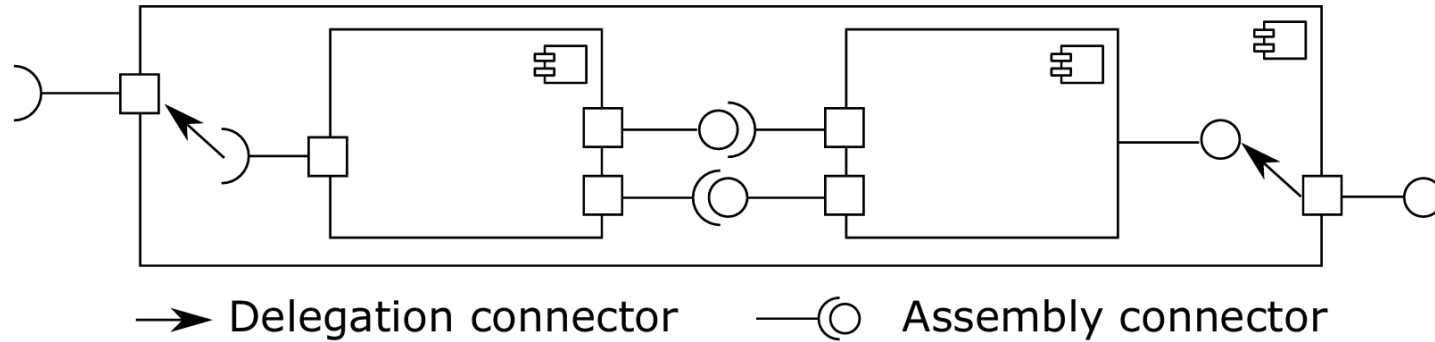


Component



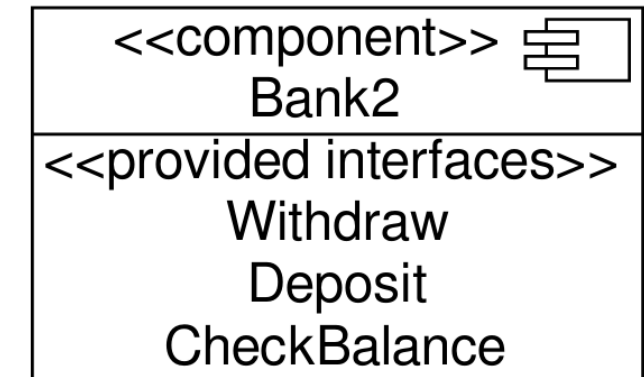
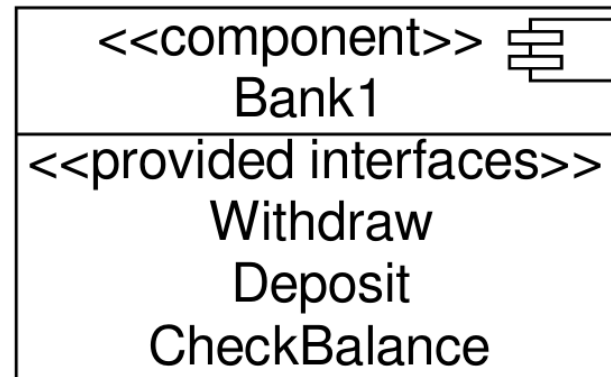
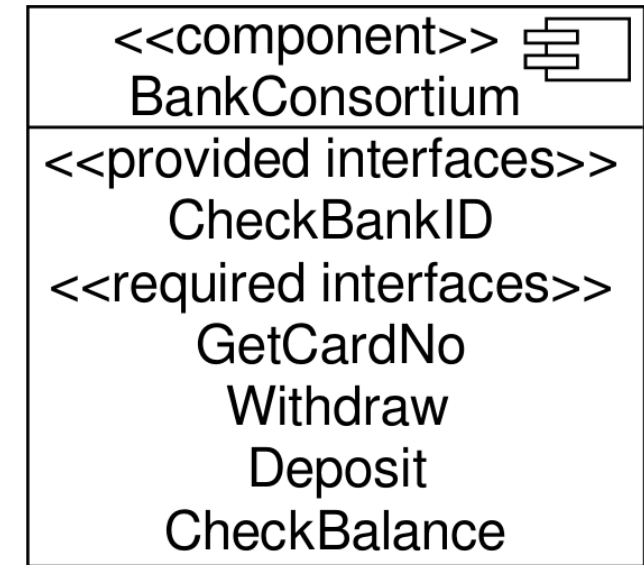
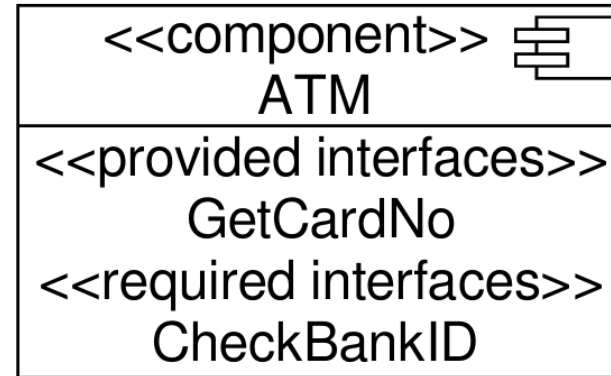
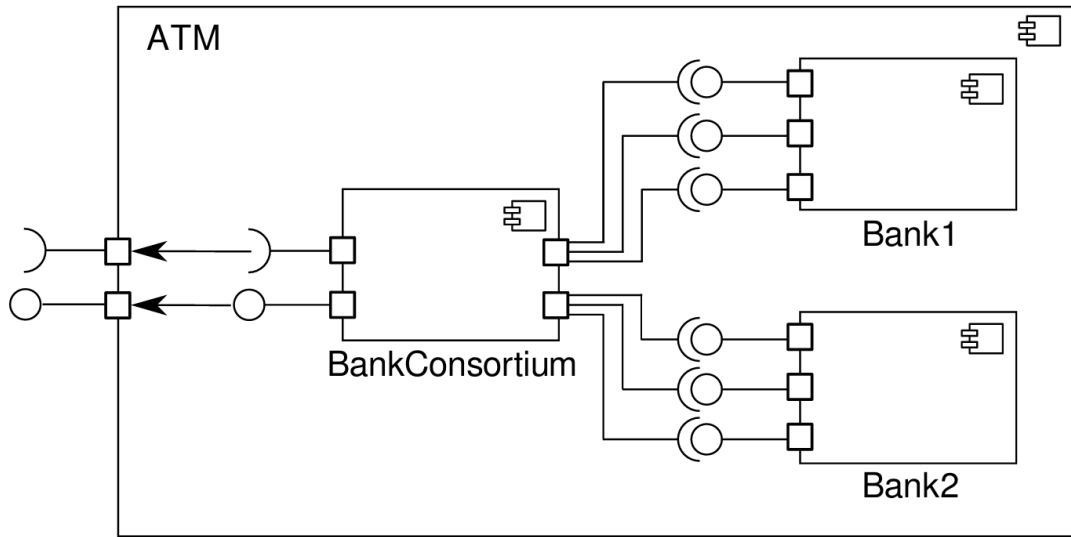
- a modular unit with well-defined interfaces by ***component-icon*** at top-right or ***<<component>>*** stereotype
- it defines services it ***provides*** and ***requires*** through its interfaces grouped into ***ports***
- ***ports*** define interaction with ***environment*** or with ***internal state (children)***
- a ***bidirectional port*** is a port with both required and provided services

5.3 UML 2.x



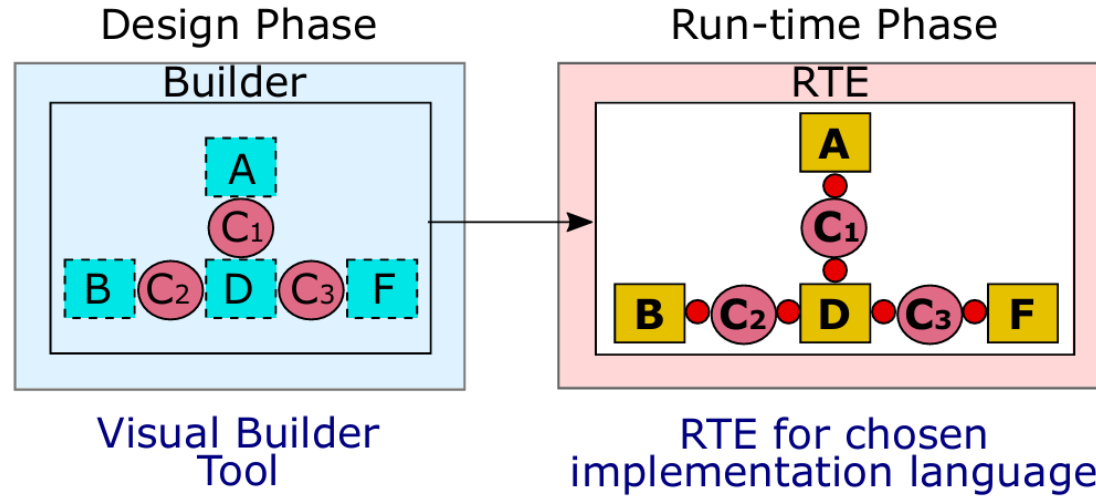
- composition is by wiring compatible interfaces in two ways
 - **assembly connectors** - match provided and required services
 - **delegation connectors** - higher level similar port matching
 - required-required/provided-provided
 - shows function/behavior of hierarchical delegation – i.e. which level sub-component actually does what action
- ex: UML 2.x ATM system and Its components

5.3 UML 2.x > Ex – ATM System and Components



- **Components** with *provided* and *required Interfaces*, with *assembly* and *delegation* connectors linking their *ports*

5.3 UML 2.x



X = UML component
X = implementation of X
C_i = UML connector
C_i = implementation of C_i
● = method call

Relation to Idealized

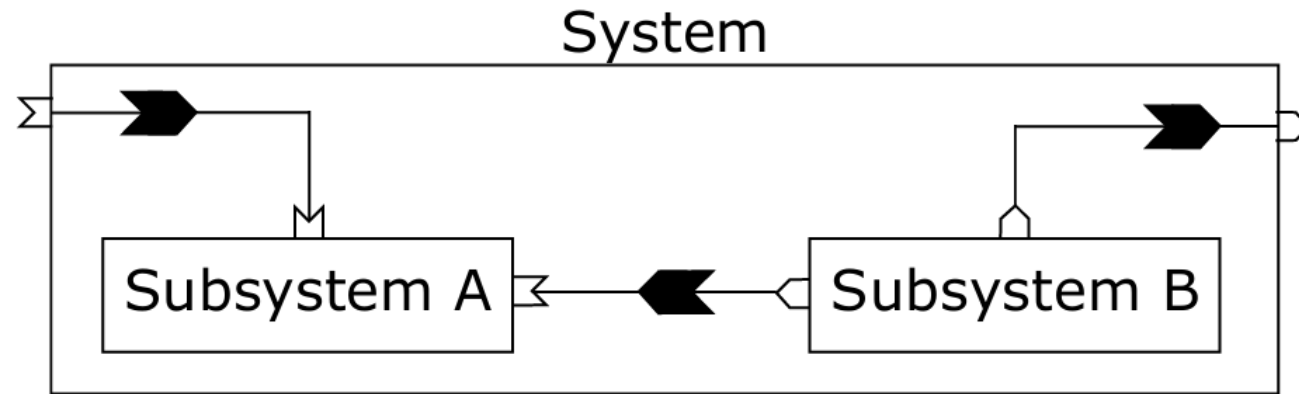
- Design Phase:
 - components are structured class diagrams - built in a visual builder like (Eclipse papyrus)
 - no implementation-code, **no repository**
 - models implemented by programmer in the language of their choice
- Deployment Phase: no **deployment phase**
- Run-time Phase: - run-time environment of Implementing language, and since UML is an object-oriented methodology (using classes) - linking is by **method call** - like ArchJava

Domain Specific Component Models

Models used for Specific purposes

5.4.1 ProCom

- a two-layered component model (***ProSys*** and ***ProSave***) intended for real-time systems, with units written in ***C Syntax***

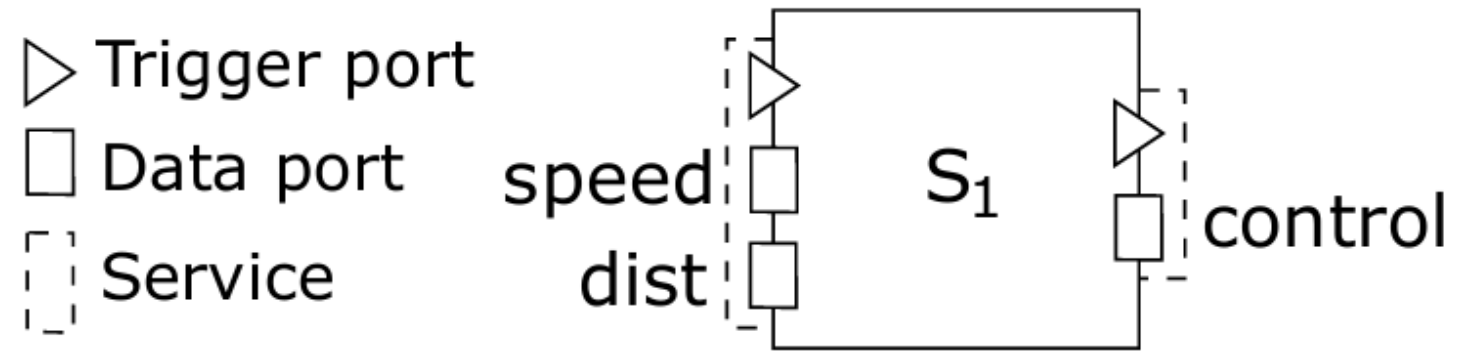


◻ Typed input message port ◻ Typed output message port ◼ Message channel

ProSys (at system level- show subsystems)

- represent ***subsystems*** containing components within the system
- they are active, distributed components with typed ***input and output message ports***
- they compose by asynchronous, explicit ***message channels***

5.4.1 ProCom > ProSave

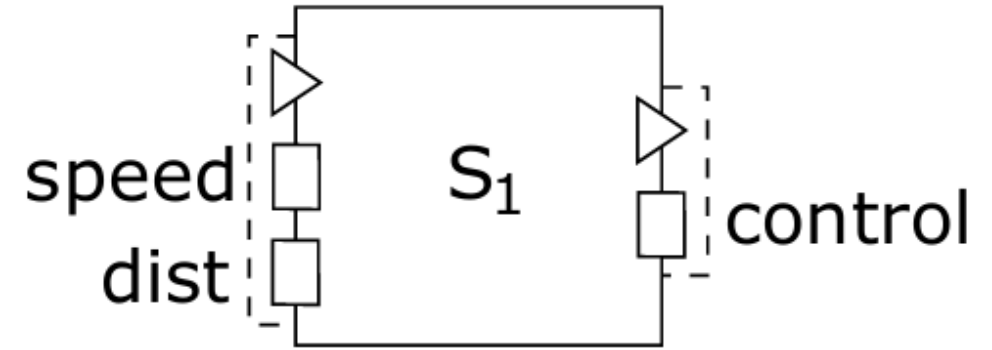


ProSave (at subsystem level)

- each ProSys subsystem is internally modeled by a ProSave component
- these are passive units of functionality encapsulating low-level tasks
- they distinguish task operation by separate ports: ***data ports*** for data-flow and ***trigger ports*** for control-flow
- functionality exposed via ***services***, and each services has
 - a group of ***input ports*** - activation triggers and required data
 - a group of ***output ports*** - provides returned data

5.4.1 ProCom > ProSave

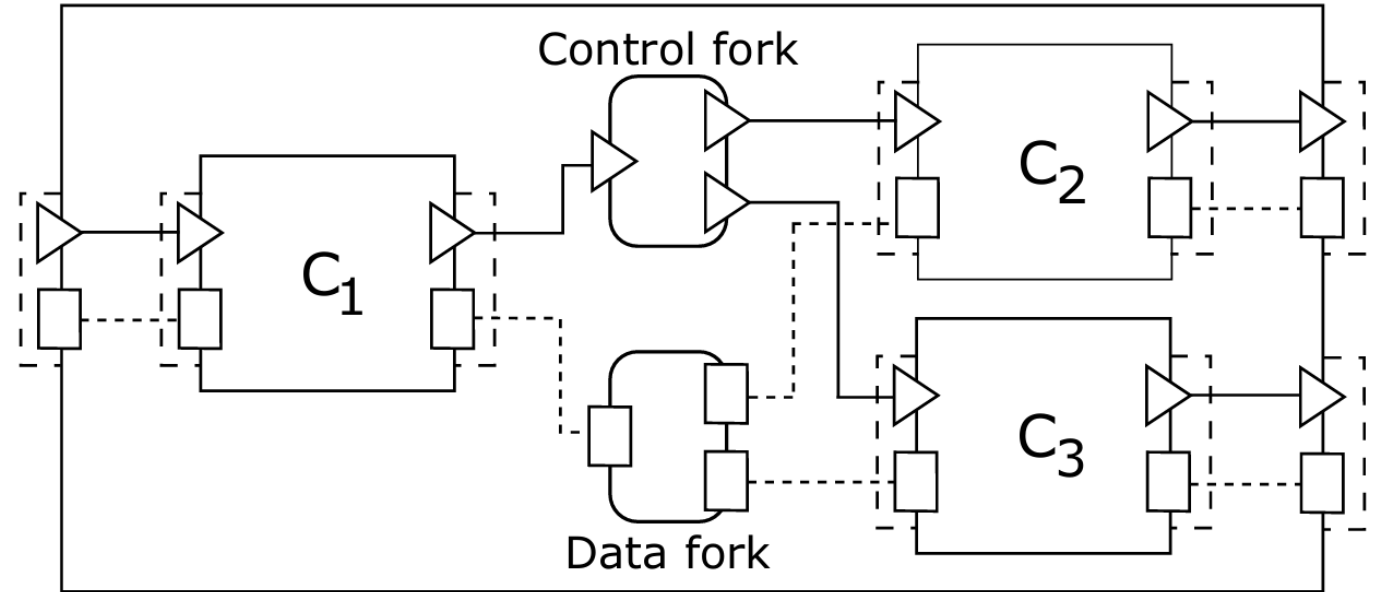
```
typedef struct{ int *speed; float *dist; } in_S1;  
typed struct{ int *control; } out_S1;  
void init();  
void entry _S1(in_S1 *in, out_S1 *out);
```



ProSave could be primitive / composite

- a **Primitive ProSave** is written code containing
 - *initializer function*
 - *C functions* implementing each service
- a **Composite ProSave** is realized by interconnecting with **connectors**
 - Trigger and data - input output matching

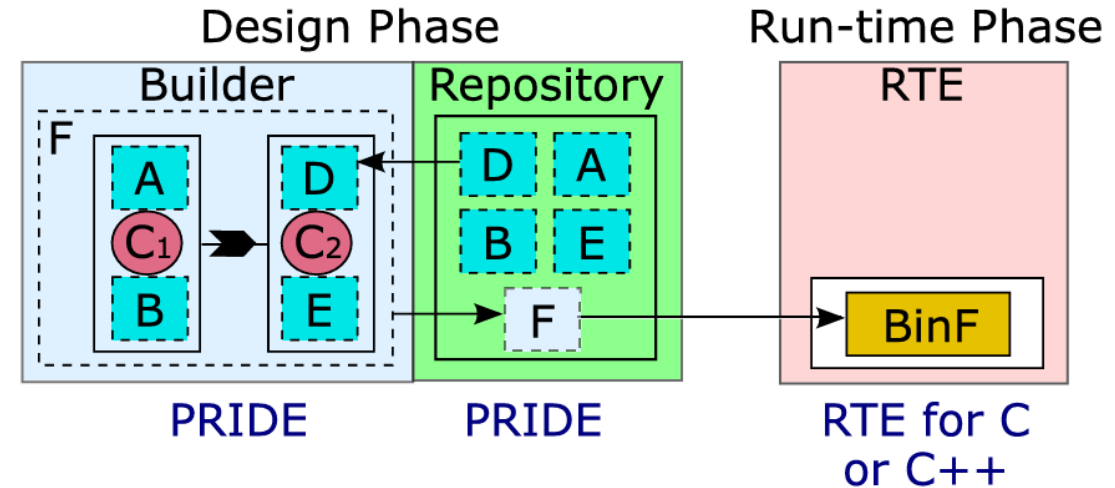
5.4.1 ProCom > Composite ProSave



- ProSave to ProSave connector types -**data fork, data or, control fork, control join, control or** and **control selection**
 - The image above shows ***control fork*** and ***data fork*** examples
- ProSave to ProSys link is done by two ways
 - **message passing** to pipe-and-filter link or
 - ***a clock for periodic activation*** of children ProSave components

5.4.1 ProCom

A,B,D,E = ProSave components
F = ProSys system
BinF = binary of F
C_i = connection
➡ = message channel



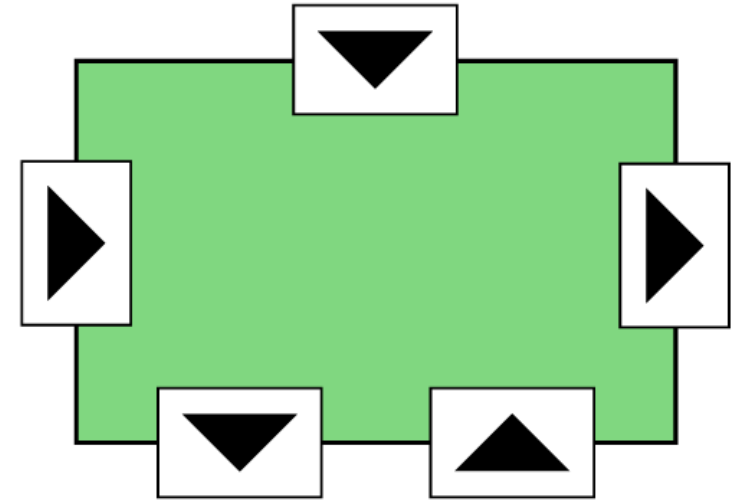
Relation to Idealized

- Design Phase:
 - ProSave components designed and built in PRIDE tool, and complete ProSys system is fully coded
 - designed components deposited in **PRIDE repository**
- Deployment Phase: ***no deployment phase***
- Run-time Phase: binary of the system is executed using implementation languages runtime environment (C/C++)

5.4.2 Koala

- a component model for describing a family of systems (related products), intended for a product-line of consumer electronics products, also written in ***C Syntax (C/C++)***

▶ interface



Component

- is a unit that interacts with other units via ***interfaces***
- an interface can be
 - ***required*** call to functions in other components or (***tip pointing outside***)
 - ***provided***, for the implemented function within the component

5.4.2 Koala

```
Component Amplifier{  
    provides VolumeControl vol;requires  
    VolumeStabilizer stb;}
```

```
interface VolumeControl{  
    void setVolume(int v);  
    int getVolume(void);  
    int a_constant = 3;  
    int a_parameter;}
```

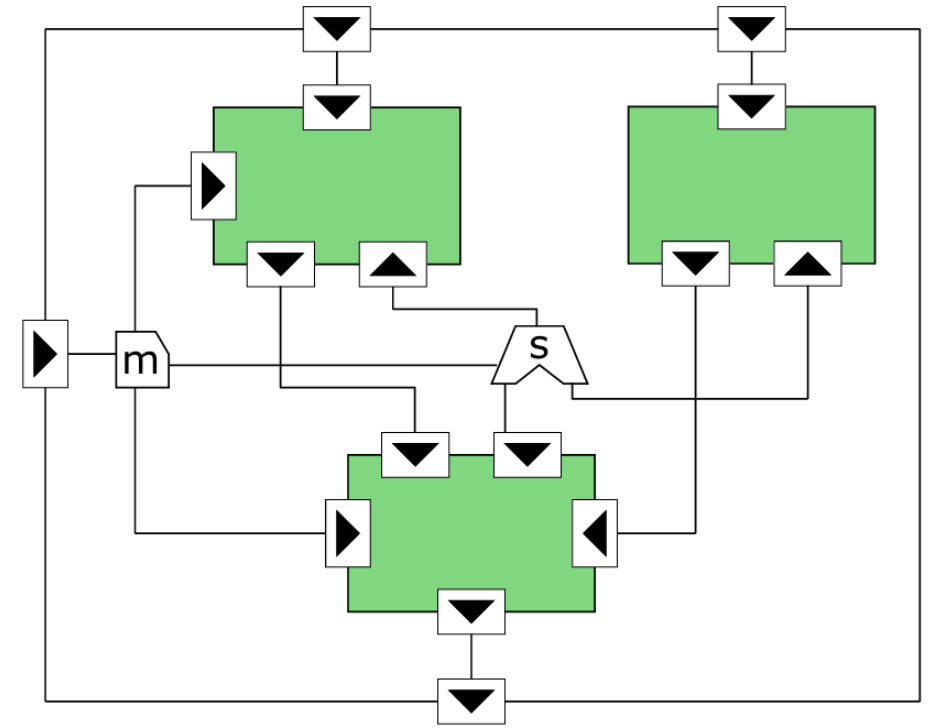
Repositories

- there is a ***global repository for Interfaces*** defined by IDL (Interface Description Language) and
- a ***global repository for Components*** defined by CDL (Component Description Language)
- ex: IDL for ***Volume control Interface*** and CDL for listing ***provided*** and ***required*** services of ***Amplifier Component***
- ***Architecturally*** all C header files used will be placed within a single directory, without external references (components are written in C)

5.4.2 Koala

 module

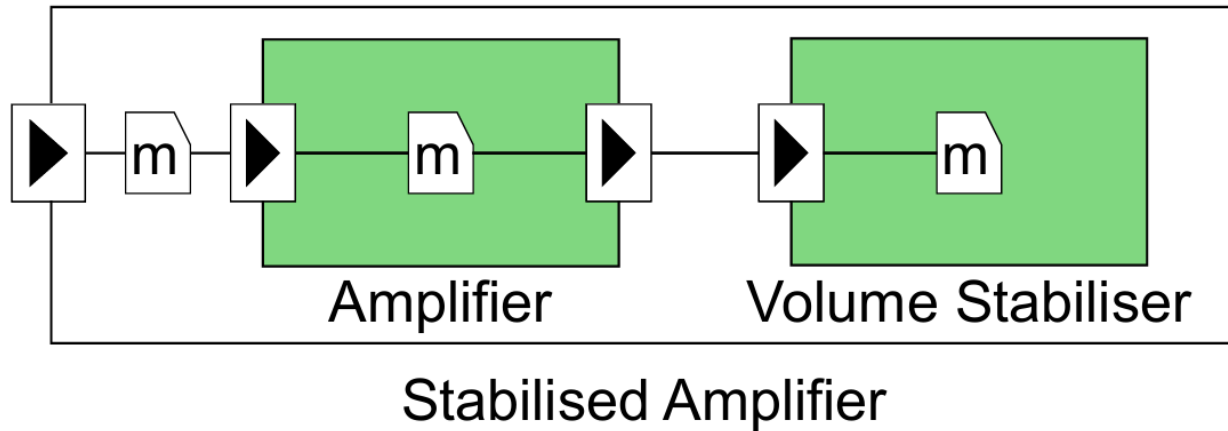
 switch



Composition Rules

- a required interface can connect to one provided interface
- a provided interface can connect to zero or more required interfaces
- **module** implements child provided interfaces and access all required to provide an interface
- **switches** reroute connection between interfaces

5.4.2 Koala > Stabilized Amplifier Example



Component Amplifier{

```
provides VolumeControl vol;  
requires VolumeStabiliser stb;  
contains module a_impl present;  
connects vol = a_impl;  
a_impl = stb; }
```

Component Stabilised_Amplifier{

```
provides StabilisedAmplifiedVolume stbAmpVol;  
contains component Amplifier amp;  
contains component Volume_Stabiliser stb;  
contains module stbAmpVol_impl present;  
connects vst = stbAmpVol_impl;  
connects amp.stb = stb.stb; }
```

Component Volume_Stabiliser{

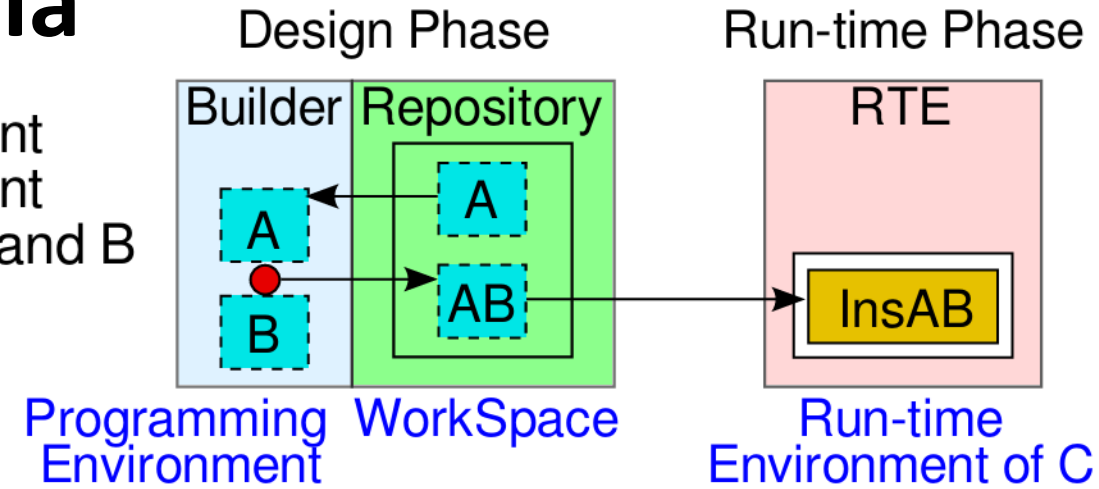
```
provides VolumeStabiliser stb;  
contains module s_impl present;  
connects stb = s_impl; }
```

Interfaces describing provided and required services and **connections** for ***connecting*** and special usage of ***contained*** module for children link

5.4.2 Koala

Relation to Idealized

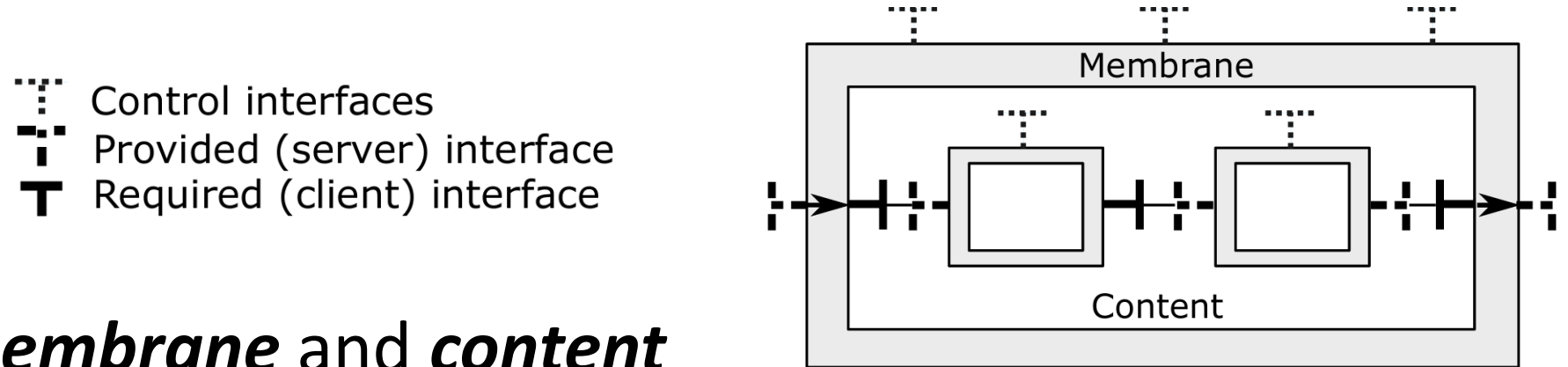
A = Koala component
B = Koala component
AB = composite of A and B
InsAB = binary of AB
● = function call



- Design Phase:
 - components (definition files) built in Koala programming environment
 - then deposited in ***KoalaModel*** Workspace (***the Repository***)
 - we can retrieve them from repository to build Systems(definition files) then deposit Systems into Repository
 - at implementation - to fully design and code completely
 - C header files (definition files) are used in writing C files and compiled into binary
- Deployment Phase: ***no further composition, no deployment phase***
- Runtime Phase: system binary code executed in C run-time env't

5.4.3 FRACTAL

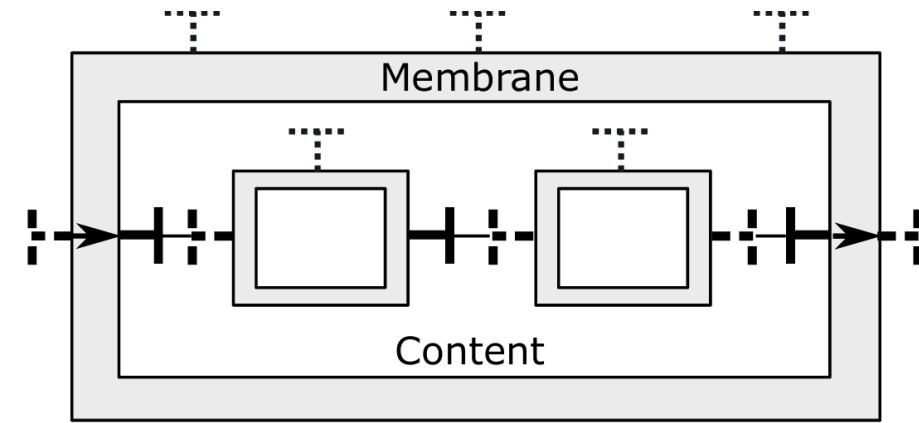
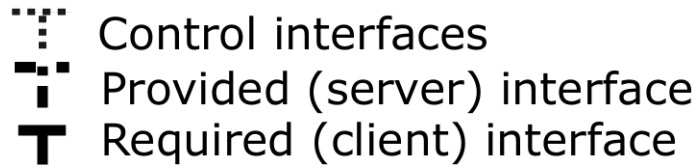
- technically still uses objects as components then provides an ADL for structuring groups of objects into Architectural Units
- objective of FRACTAL is to enable modular development with reconfigurable OOP systems i.e. enabling ***Introspection*** for identifying and reconfiguring Internal state of components



Component: has ***membrane*** and ***content***

- ***membrane*** gives external **interfaces** (provided, required and **Control**): **control** is for **configuring** and **introspecting** its content
- ***content*** consists of a finite set of ***sub-components***

5.4.3 FRACTAL



Levels of Control on FRACTAL components:

➤ **No Control:**

- It is not a must to have controller interfaces on FRACTAL components,
- comparable POJO's with no introspection and interception capabilities

➤ **Elementary Control:**

- Interface implemented for structuring (like IUnknown in COM)
- ADL written in XML provided to represent control Architecture

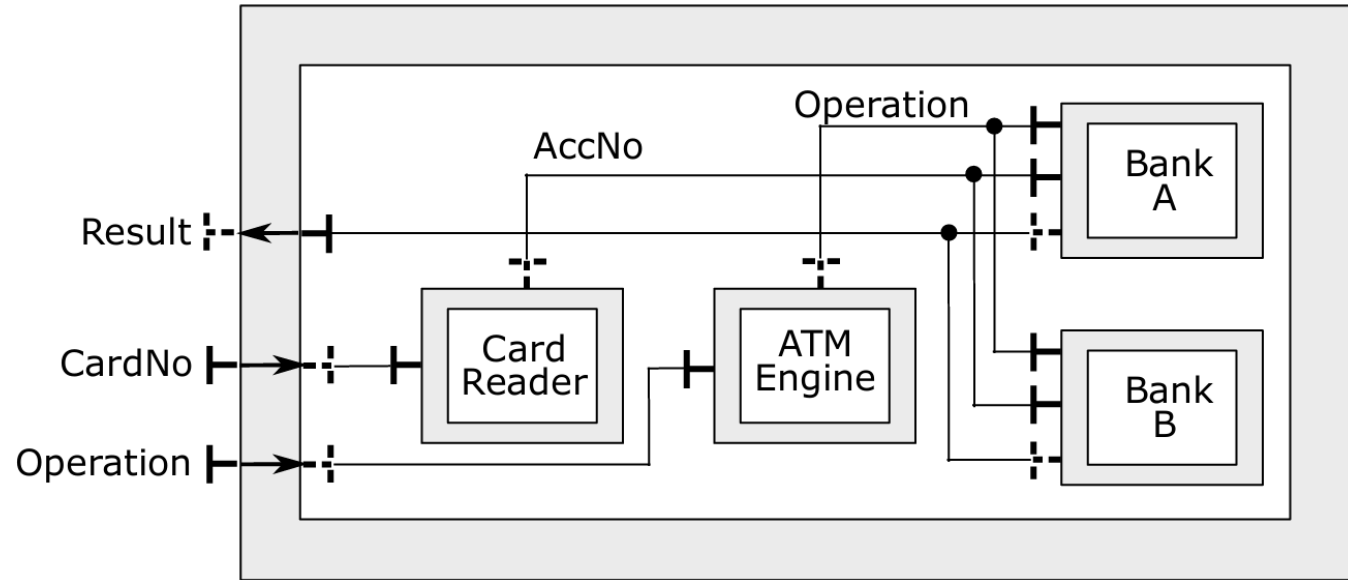
➤ **Advanced Control:**

- Combines and extends multiple controllers to yield custom control
- Some controller are: ***attribute, binding, content, life-cycle*** controllers

Composition: component can be ***primitive*** or ***composite***

- **primitive:** has no sub-components, just interfaces and defining class
- **composite:** defined by sub-components and interface-binding

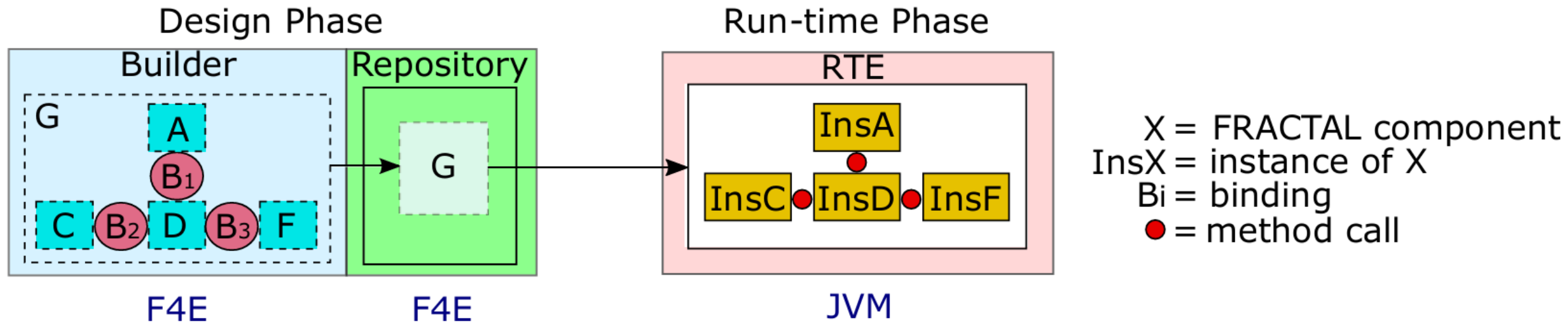
5.4.3 FRACTAL > ATM Example



Composition: component can be *primitive* or *composite*

- **primitive:** has no sub-components, just interfaces and defining class
- **composite:** defined by sub-components and interface-binding

5.4.3 FRACTAL



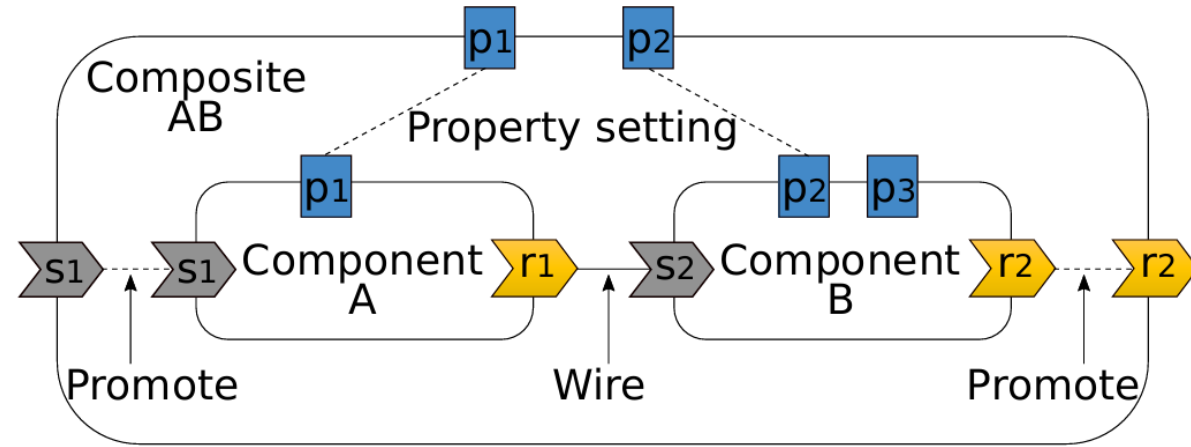
Relation to Idealized

- Design Phase
 - designing components and composite architecture by interface binding
 - FRACTAL for Eclipse (F4E) IDE tool - ***(deposit only repository)***
- Deployment Phase: no assembly, thus **no deployment phase**
- Run-time Phase: Run on Java Virtual Machine (JVM)

Other Models

Replaced, but for Historical Context

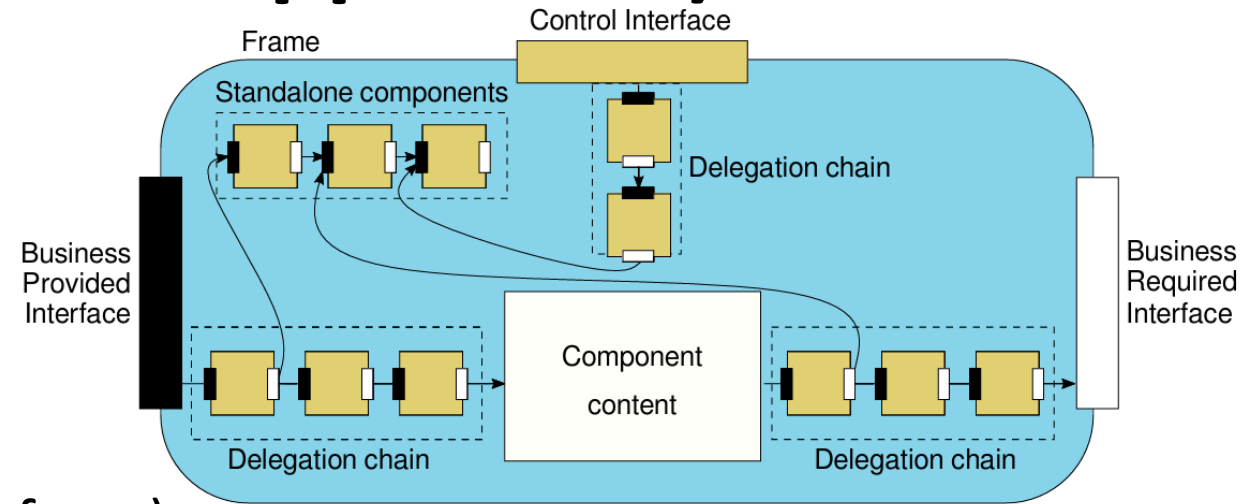
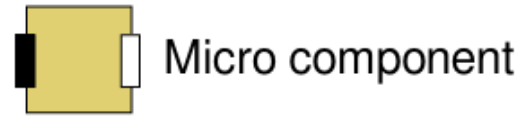
5.5.1 SCA (Service Component Architecture)



Model for composing applications following **Service-Oriented Architecture (SOA)** principles

- aims for **programming language and application environment neutrality**.
- **Components**
 - have (**Services, References, Properties, Bindings**)
 - they implement business logic exposed as **services**
 - rely on services from other components through **references**
- **Composition**
 - Components are composed into composites by **wiring** their **services** and **references** - link is specified by **binding**.

5.5.2 SOFA (SOftware Appliances)



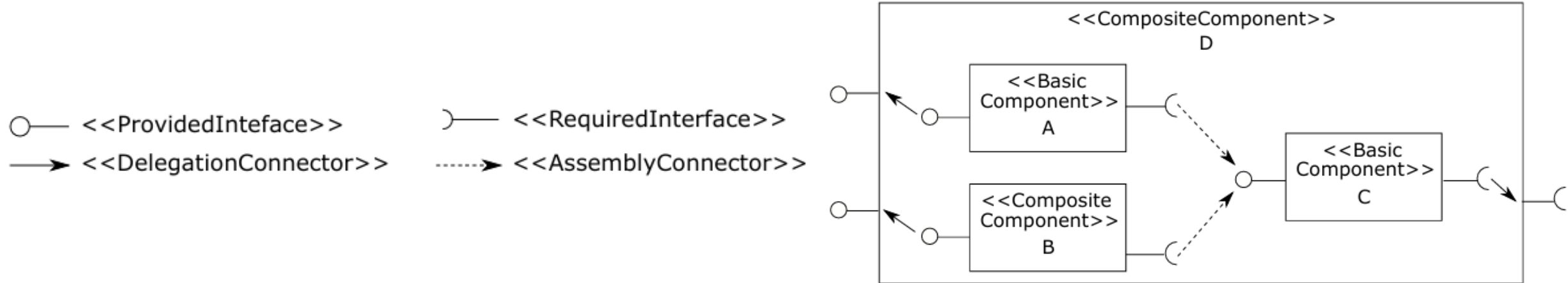
Component

- they are described by
 - **frame** (black-box view with business interfaces)
 - **architecture** (implementation with sub-components and interconnections).
- have **control interface, provided interface, required interface, content (child)**,
- They also includes **micro-components** and **SOFAnodes** with repositories.

Composition:

- SOFA components are composed by **binding their interfaces** together using **connectors**.
- SOFA 2 distinguishes between **design and runtime connectors**,
 - **design connectors** specifying the communication style and
 - **runtime connectors** implementing them.

5.5.3 Palladio Component Model (PCM)



- Built to enable **quality of service (QoS) prediction**, especially performance and reliability, for component-based architectures described by ADLs.

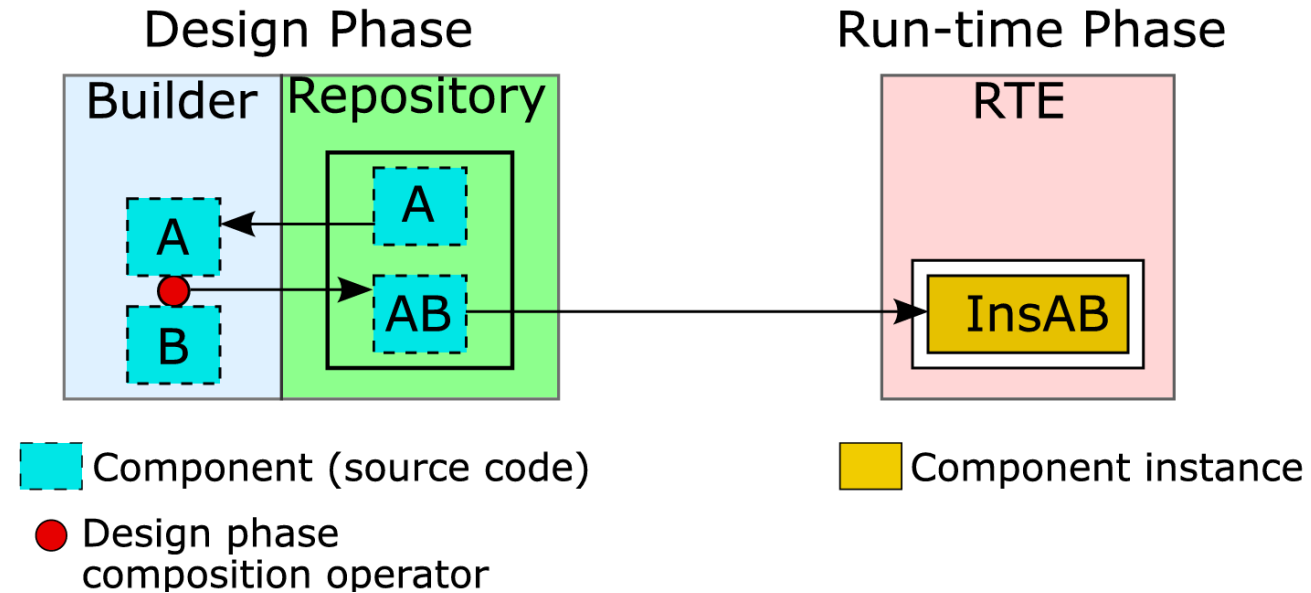
Components

- are architectural units with provided and required interfaces,
- they are composed by **assembly and delegation connectors**.
- **Component Types** (provided, complete, implementation) and **Service Effect Specification (SES)** specifies the behavior of a component.
- PCM also involves different **Roles and Models** for various stakeholders.

Composition:

- **assembly connectors** to link required and provided interfaces and
- **delegation connectors** to forward interfaces of sub-components to the composite.

5.5 SCA, SOFA & Palladio > Idealized C. lifecycle



Relation of SCA, SOFA & Palladio to Idealized Component Lifecycles - Commonalities

- Design Phase:
 - builder tool to build and deposit new components to repository
 - retrieve, compose and redeposit composites into repository
 - until depositing complete systems
- Deployment Phase: no further composition - ***no deployment phase***
- Run-time Phase: same composition as finished in design phase

Thank You!